

## **\*\*Chapter X Personal Context Experiences\*\***

**\*\*I still remember the morning I found it.\*\***

**\*\*I remember waking up that morning feeling like something was off, and remember having this feeling like "something, sometime today doesn't matter if at work, at home, or wherever - that something bad is going to happen. It gave me morning anxiety like crazy. I arrive to work, put my bag down at my desk, and like clockwork, after I place my belongings at my desk, I head straight into the 60 degree datacenter and do a twice-walkthrough of the server rack aisles doing a visual inspection, making sure I see no amber or red lights, nothing is beeping, or buzzing for attention, check the humidity and temperate controls on the huge Lieberman System, made sure no random cut or spliced pieces of network cable were in walking paths, just doing a visual inspection of the overall health of the data center was always my first start of business even before I sat at my desk and logged into email.\*\***

**\*\*It was one of those quiet days in the SCIF - the kind where the hum of the racks seemed louder than the people I could hear a couple of server rack aisles down. That week, I was knee-deep in a privileged access audit for an upcoming FISMA audit, looking and clicking through ADUC groups and OUs I'd seen a thousand times before.\*\***

**\*\*That's when I saw something I didn't think I had initially seen. I had to re-enter the group account to verify I wasn't hallucinating.\*\***

**\*\*An account named something like OpsAdmin-Legacy.\*\***

**\*\*Since I was primarily in charge of commissioning/decommissioning everything in AD from user and group account creation, to Exchange and Outlook email profile creation, to assigning user and group permissions on network file shares, we had a graveyard OU of old accounts that had been disabled and in the notes field, the tech's initials and date of when the account modification occurred for accountability reasons.\*\***

**\*\*Something about that group account I did a double-take on made me pause. Maybe it was the "legacy" part. Maybe it was the fact that I'd never seen it used in any of the systems I managed.\*\***

**\*\*So I opened it.\*\***

**\*\*And my stomach dropped (as I kicked myself in the ass for not having found it sooner).\*\***

**\*\*This thing wasn't just an old application admin account. It was a shared admin account with Domain Admin rights. Not just DA - it had local admin everywhere, GPO and OU write permissions, and the worst part: unconstrained delegation flag was present. It was basically a skeleton key to our entire domain.\*\***

**\*\*And nobody had touched it in months.\*\***

**\*\*It felt like I found a loaded weapon in someone's desk drawer.\*\***

**\*\*How It Got There\*\***

**\*\*I started digging through our ServiceNow closed troubletickets assigned to every admin, old project folders, old emails looking for information that would validate why the account was made and what and whose used it and what it was used for.\*\***

**\*\*Eventually the store came together:\*\***

**\*\*Years ago, contractor team had used the account for an SCCM migration project. They needed elevated access \_temporarily.\_ The project ended. The contractors rolled off, and the account stayed as it was.\*\***

**\*\*No one removed the permissions.\*\***

**\*\*No one disabled it.\*\***

**\*\*No one put their initials and timestamp into the notes field after having been repeatedly told to do so for risk tracking and accountability sake.\*\***

**\*\*It was the classic government IT tale - temporary access that, over the year or two, quietly fossilized into permanent access because no one wanted to break something.\*\***

**\*\*I noticed it, but only because I was doing a manual line-by-line review instead of trusting our HBSS ABM monitoring dashboards.\*\***

**\*\*It didn't show up in any of the privileged access report generated nightly because:\*\***

- **\*\*It wasn't tied to a specific individual\*\***
- **\*\*It wasn't in the PAM vault\*\***
- **\*\*It wasn't tagged as being privileged\*\***
- **\*\*It didn't require mandatory PIV authentication\*\***
- **\*\*It wasn't in our CMDB\*\***

**\*\*All these things in and of themselves made me even more nervous because that led me to believe and think about there maybe being a bigger issue at hand and thought "what hole is open wide enough that someone could have done this?"\*\***

**\*\*It was a literal ghost account, and one with god-mode permissions.\*\***

**\*\*What Could Have Gone Wrong Had I Not Spotted the Account\*\***

**\*\*Well, everything, frankly.\*\***

**\*\*If an adversary had found it, they could have easily pwned the domain within mere minutes (given the attacker was skilled, had aptitude, and knew what they were doing). Harvested tokens. Modified everything. Disable logging. Spread malware. And**

because it was a shared account, no one would ever truly know who did what.\*\*

\*\*We shut it down immediately like any good defender would do - disabled the account, pulled logs, tore out permissions, made images of the breached operating systems for attack entry analysis and\*\* then turned the computer off.

But the real lesson here wasn't just the cleanup.

It was this: \*\*The most dangerous accounts in a federal environment aren't the ones attackers create. They're the ones we forget.\*\*

\*\*2\. Service Accounts Nobody Owned\*\*

\*\*I once inherited a domain where the service accounts felt like ghosts - everywhere, touching everything, and nobody alive and breathing could tell you exactly who created them or why; however, one account in particular still stands out in my mind.\*\*

\*\*It was named `svcDataSync01`, which sounded harmless enough. We had dozens of accounts like that. But during another routing privileged access review, I noticed it had logged in recently...despite the application it supposedly belonged to being decommissioned over two years ago.\*\*

\*\*That was the first red flag.\*\*

So I opened the account properties in ADUC, and the second instant red flag hit me like a brick: the password was set to never expire.

The last password change was nine years ago.

Nine.

In federal AD time, that's considered prehistoric.

I asked around - the app team, the DBAs, the malware team, even the old-timers who'd been there since the NT4 days. Every conversation went the same way:

"Do you know what this account does?"

\_No idea.\_

"Is it tied to anything critical?"

\_Probably? Maybe? Hard to say.\_

"Can we disable it?"

\_Nooo - what if it breaks something?\_

That was the third red flag: fear.

Fear is the natural habitat of bad service accounts.

So I decided to dig deeper.

And that's when I realized how bad it really was.

This seemingly harmless service account had:

- Local admin rights on several servers
- Write access to a shared file system used by multiple mission apps
- Logo as a service on a critical legacy system
- Interactive logon enabled (why? Nobody knew)
- Membership in a group that had been granted GPO edit rights

It was the classic example of the definition of **\*\*over-permissioned.\*\***

And the best part?

No one could tell me what would happen if I disabled it.

Not one person.

Not even my direct supervisor whose only advice for me was to "just figure it out."

So I did.

It was tied to a legacy data-exchange process that had been rewritten three different times, migrated twice, and patched endlessly. Somewhere along the way, the documentation evaporated, the original engineers retired, and the service account just... stayed.

A ghost with keys to half the kingdom.

We were stuck in the classic federal IT paradox:

- If we leave it alone, it remains a security risk we have to be accountable for.
- If we disable it, we chance breaking something critical and possibly face repercussions from leadership.

So we did what any cautious operator would do: we built a plan.

We monitored its logons.

We traced its Kerberos ticketing activities.

We watched what files it touched.

We mapped its group memberships.

We checked every scheduled task, every service, every batch job.

And after two weeks of watching it like a hawk, we finally realized the truth:

It wasn't doing anything.

Nothing.

Not a single meaningful action.

It was a zombie - a relic from a system that no longer existed.

So we disabled it.

And nothing broke.

Not a single alert.

Not a single ticket.

Not a single user complaint.

It was like cutting a wire that wasn't connected to anything except you couldn't see what was on the far-end.

But the lesson stuck with me:

The most dangerous service accounts aren't the ones doing too much. They're the ones doing nothing - but still have the power to do everything.

3\. The Access Review Nightmare - here's a story about an access review \_that looked perfect on paper\_ but was an absolute disaster in reality - the kind of thing every federal NetSecOps operator has seen at least once.

Every year we did the same ritual: the quarterly access review.

On paper, it was supposed to act as our safety net - the process that caught stale accounts, excessive permissions, and anything that didn't belong in a FICAM-aligned environment.

In reality?

This one was a masterclass in how an access review can fail while technically "passing."

It all started with a rushed deadline. Never a good thing.

We were already behind schedule because the review packet had bounced between three different offices. By the time it landed on my desk, leadership wanted it done \_yesterday.\_

Managers were told to "review and approve" their staff's access.

Except most of them didn't understand half the groups listed.

Some approvals came back in under two minutes.

Two minutes to review dozens of permissions across multiple systems.

That was the first sign that the review was going to be a nightmare.

**\*\*Managers rubber-stamped everything.\*\***

I watched one supervisor scroll through the entire list of users and groups, nod once, and click "Approve All."

I asked him if he recognized every permission. He smugly shrugged and said:

\_"If they had it last quarter, they probably still need it."\_

That's when I knew the review was meaningless to many.

**\*\*Roles were vague to the point of comedy.\*\***

We had groups named things like:

- AppSupport-TierX
- Legacy-Ops-Admin
- SpecialAccess-West
- DataTeam-Ext

No one could explain what the "Ext" meant.

No one knew what "TierX" was.

And "Legacy-Ops-Admin" was apparently "important," but no one could state why.

When roles are vague, people approve out of fear and hastiness - not understanding.

**\*\*Old users were still active.\*\***

During the review, I noticed:

- A contract who left 14 months ago still had VPN access
- A vendor account with no expiration date
- A user who transferred to another agency, but still had access to our SharePoint farm
- A retired GS-14 who somehow still had mailbox access

Every one of them was approved by their manager, even though none of them worked

here anymore.

When I asked why, the answer was always the same blanketed one:

\_"I thought someone else was handling that."\_

The review completely missed privileged access.

This was the part that made my blood pressure spike.

Buried in the list was a user who had:

- Local admin on 200+ workstations
- Write access to a GPO
- Membership in a group that granted server operator rights
- A role that allowed interactive logon to a sensitive enclave

And their manager approved it without blinking.

When I asked him why the user needed those permissions, he said:

\_"I don't know - they've always had them."\_

That's when it hit me: the review wasn't about validating access. It was validating history.

The moment everything fell apart was when the ISSO asked for justification for one of the privileged groups. The manager who approved it said:

\_"I didn't know that was privileged."\_

Another manager admitted he didn't even open the spreadsheet. One said he approved everything because he \_"didn't want to break anything."\_

Another said the permissions "looked too technical" so he assumed they were necessary for the continuation of mission-processes.

The review passed on paper.

But in reality, it failed every control it was implemented and meant to enforce.

The lesson:

A rushed access review is worse than no review at all. Because it creates the illusion and gives people a false sense of security while leaving all the real risks untouched.

Rubber-stamping isn't governance.

It's negligence disguised as compliance.

4\. The Time a Password Reset "Fixed" Nothing - here's a story about one of the most frustrating patterns in federal NetSecOps - when people think that just by resetting a password means the same thing as fixing the actual problem at hand.

This one still makes me shake my head.

It started with an alert: suspicious activity tied to a mid-tier application account. Nothing dramatic - just enough noise to make us look twice. The help desk's first instinct was the classic move:

\_"We'll just reset the password. That should take care of it."\_

And before anyone in NetSecOps could blink, they did.

Password reset complete. Ticket closed. Problem solved. Move forward.

Except... nothing was solved.

The real issue wasn't the password at all.

The account in question wasn't a normal user. It was yet again another service account - one that had been around longer than half the staff. And the real problem wasn't credential theft.

It was everything else:

- It was a member of three privileged groups no one remembered creating
- It had write permissions on a shared directory used by multiple mission apps
- It had interactive logon enabled
- It had cached credentials on several servers
- It was used by a scheduled task running under the old token
- It was tied to a legacy process that nobody wanted to touch

Resetting the password didn't fix any of that.

The account still had way too much access.

After the reset, the account still had:

- Local admin rights
- Access to a sensitive SQL instance
- The ability to modify a GPO
- A Kerberos token that was still valid on one server
- A running service using the old password that kept retrying authentication

So the "fix" actually created more noise, not less.

**\*\*There was zero follow-up validation.\*\***



No one checked:

- Whether the account still had privileged group memberships
- Whether the services using it were updated
- Whether the cached tokens were invalidated
- Whether the ACLs were written appropriately
- Whether the account even needed to exist

The help desk closed the ticket and moved on.

Management only saw "password reset" and assumed the threat was neutralized.

Meanwhile, the account was still a standing security hazard.

**\*\*What people misunderstood:\*\***

They thought the problem was the password itself.

They failed to understand:

- Group membership is often more dangerous than a password by itself
- ACLs grant silent, persistent access
- Tokens don't magically disappear when you reset a password
- Service accounts do not behave like user accounts
- Shared accounts can't be secured with a reset
- Processes using old credentials will keep trying to authenticate
- Privilege doesn't evaporate just because the password was changed

Resetting the password was like repainting a cracked dam.

How it finally got fixed was when NetSecOps got involved. They:

- Mapped every permission the account had
- Identified every system using it
- Removed it from privileged groups
- Replaced it with a properly scoped service principal
- Updated all services and scheduled tasks
- Cleared cached credentials
- Validated no tokens were still active
- Documented the entire chain for the ISSO.

Only then was the issue actually resolved.

The lesson: a password reset is not remediation. It is a band-aid - and sometimes a dangerous one.

The real threat usually lives in:

- Group memberships
- ACL misconfigurations

- Service account sprawl
- Cached tokens
- Shared accounts
- Privileges creep

Resetting a password doesn't fix any of these.

5\. Here's a story about one of the worst shared admin account situations I ever saw - the kind that makes every NetSecOps operator's eye twitchj. It's the perfect example of why shared admin accounts are a security nightmare to deal with, especially in a federal AD + FICAM environment.

There was an account in the domain called something like AdminOps-Shared. The name alone should have been my first red flag, but it had been around so long that we all treated it like part of the office furniture - ugly, outdated, but too much trouble to replace.

I first stumbled onto it during an audit prep. The logs showed it authenticating from multiple locations within a few minutes. That's physically impossible for a single user, so I started asking questions like I always do.

That's when the truth was revealed.

I found out it was shared years ago, before FICAM alignment, the operations team created it as a "temporary workaround" so multiple admins could manage a legacy system that did not support role-based access. The plan was to replace it within a month.

That month turned into a decade.

Every time someone suggested retiring it, the answer was the same.

\_"We'll fix it after the next upgrade."\_

Spoiler: that next upgrade never came.

At least 12 people had the password.

Probably more - because the password had been emailed around, written on sticky notes, and passed verbally to contractors who rotated in and out.

One person even admitted: \_"I think the vendor knows it too... or at least they used to."\_

That was not comforting to hear.

When I pulled the logs, all I saw was:

**\*\*AdminOps-Shared loggedon\*\***

**\*\*AdminOps-Shared modified GPO\*\***

**\*\*AdminOps-Shared created a new local admin\*\***

**\*\*AdminOps-Shared accessed a sensitive file share\*\***

But \_which human\_ did those things?

No idea.

There was no really way to tie those actions to any one individual.

No non-repudiation.

No auditable trail.

No accountability (which I'm big on)

It was, in fact, the exact opposite of what FICAM requires.

MFA was missing, and because it was a shared account, it couldn't be tied to a PIV card. So it was password-only.

No MFA.

No certificate.

No smart card.

No assurance.

Just a static password protecting domain-level access.

The password was stored somewhere unsafe, and this part still makes me cringe: The password was:

- Written on a whiteboard in a server room for all to see in plain view
- Saved in a plaintext Notepad file on a shared drive
- Included in an old onboarding document
- Mentioned in a ticket from 2017
- Known by at least one contractor who left years ago

It was basically public knowledge at that point.

Logs only showed the shared account, and that was the worst part.

During an incident investigation, we needed to know who made a specific change to a GPO. The logs showed:

**\*\*AdminOps-Shared modified policy X at 0214.\*\***

That was it.

No workstation ID.

No user SID.

No correlated event to tie into.

No way to really know for sure if it was a legitimate admin or an adversary.

We had to interview every admin on the team like it was a crime scene.

Everyone said the same thing: `"It wasn't me."`

And maybe they were telling me the truth.

Or maybe they weren't.

I had no real way of really knowing.

Shared admin accounts don't just break best practices - they break accountability, auditability, identity assurance, and incident response.

They are the perfect hiding place for:

- Insider threats
- External attackers
- Misconfigurations
- Mistakes
- Privileges abuse

And the worst part? They give you the illusion of control while removing the ability to prove anything.

## 6\ The Documentation Said One Thing - Reality Said Something Else

Here's a story about one of the worst documentation mismatches I ever saw - the kind that makes you question whether the environment was built first or the paperwork was.

We were prepping for an upcoming audit, and leadership proudly handed us a pristine binder: the System Security Plan, network diagrams, SOPs, access matrices - the whole polished package.

It looked perfect.

Too perfect, in fact.

The diagrams were clean.

The roles were clearly defined.

The access matrix looked like it came straight out of a textbook.

But the moment we compared it to the actual environment, the whole thing fell apart.

What was wrong about it was pretty much everything.

The SSP claimed:

- The system used role-based access control (RBAC)
- All privileged accounts were PIV-enforced
- Service accounts were documented and reviewed annually
- The network had two enclaves, clearly segmented
- Only three admin groups existed
- All legacy systems had been retired

However, in reality:

- Roles were vague, overlapping, and inconsistent
- Half the privileged accounts were password-only
- Service accounts were a graveyard of unknowns
- There were actually five enclaves, two of which no one admitted to owning
- There were 17 total admin groups, many nested inside each other
- The "retired" legacy system was still running in a closet rack

The documentation wasn't just outdated - it was **\*\*fiction.\*\***

It was outdated, incomplete, copied, and inaccurate.

- Parts of the SSP were copied from another system
- The diagrams were five years old
- The access matrix was incomplete
- The SOPs referenced tools we didn't even use anymore
- The policy section described controls that had never been implemented once
- The architecture diagram still showed a firewall that had been removed during a migration

It was like reading a biography of someone who never existed. None of it mattered.

But, leadership trusted the document anyway. That was the surreal part for me.

Leadership waved that binder around like it was gospel.

They kept saying things like:

\_"The documentation shows we're compliant."\_

\_"The auditors will love this."\_

\_"Everything is already approved in the SSP."\_

They trusted the paper more than the actual environment we were living in.

When we pointed out discrepancies, the response was;

\_"If it's in the SSP, then that's how it works."\_

Except... it wasn't.

This risk turned into a massive risk.

- False sense of security

Everyone believed the systems were locked down because the paperwork said so.

- Privileged access hiding in plain sight

The access matrix didn't list half the admin groups, so no one reviewed them.

- Incident response blind spots

The network diagram didn't show the actual data flows, so IR teams would have been chasing ghosts.

- Compliance exposure

If an auditor had compared the SSP to the real environment, we would have failed instantly.

The whole package presented operational risk. The undocumented enclave had direct access to a mission-critical system - and no one was monitoring its activities.

Identity governance collapsed as roles did not match permissions, permissions did not match groups, and the groups did not match or reflect reality.

It was the perfect storm: documentation that looked compliant at first glance, and an environment that wasn't.

The lesson I learned was that security documentation isn't supposed to be aspirational. It's supposed to reflect actual reality - even when reality is messy.

When the paperwork says one thing, and the environment says another, you don't have governance. You have fiction masquerading as compliance.

## 7\ The "Temporary" Exception That Became Permanent

Here's a story about one of the most memorable "temporary" exceptions I ever saw - the kind that was supposed to last a week but ended up haunting the environment for years. It's the perfect example of how a quick workaround becomes a long-term

liability in a federal NetSecOps world.

It started during a high-pressure deployment weekend. A mission-critical application wasn't talking to a backend database, and leadership was breathing down everyone's neck. Classic fingerpointing went on: the apps team insisted it was a firewall issue. The network team insisted the rules were correct, and that it was a coding error on the database team's side. The database team insisted nothing had changed on their end.

So someone - I still don't know who - proposed a "temporary" fix.

\_"Let's just open the firewall between the app subnet and the DB subnet for now. We'll lock it down after the deployment."\_

Everyone nodded.

Everyone agreed it would be temporary.

Everyone promised to revisit it come Monday.

You already know where this story is going.

Why the exception was created? Pure urgency.

The system was tied to a mission timeline, and leadership wanted it online immediately. The exception was created to "unblock the deployment" and to "buy time to troubleshoot later."

Later never came.

Who approved it?

Technically? A senior manager who didn't fully understand the implications.

Realistically? The room approved it through collective panic.

There was no formal waiver, no risk acceptance memo, no ISSO sign off- Just a verbal "do it" from someone with enough authority to override the entire process.

Was there an expiration date? Yes - on paper. The ticket said:

"Temporary rule. Expires in 7 days."

But the expiration wasn't tied to an automated process.

It wasn't tracked in a dashboard.

It wasn't added to a review queue.

It was just a plain comment in a ticket that no one bothered to reopen.

Did anyone track it? No.

The firewall team assumed the app team would request removal.

The app team assumed the firewall team would solve it automatically.

The ISSO assumed the exception had been documented somewhere.

Everyone just assumed.

No one actually verified.

What made it risky? Everything.

- The rule was overly broad

Instead of opening a single port, they opened ANY-ANY between two sensitive enclaves.

- It bypassed segmentation

The entire purpose of the enclave design was undone by one rule.

- It created a lateral movement highway

If an attacker compromised the app server, they had a straight shot into the database tier.

- It wasn't logged as an exception

So it never appeared in quarterly reviews.

- It normalized bad behaviors

Once people saw how easy it was to get a "temporary" exception, they asked for more.

### How It Was Discovered

Two years later - yes, years - we were doing a network segmentation audit. I noticed a rule that made no sense:

```
**AppSubnet > DBSubnet: ANY-ANY**
```

I asked the firewall team about it. They shrugged.

I asked the app team. They had no idea.

I asked the DBAs. They said the app didn't even use that path anymore.



It was a ghost rule - a leftover from a crisis no one remembered.

The Fallout

When we finally removed it, nothing broke.

Not a single alert.

Not a single ticket.

Not a single user complaint.

It had been unnecessary for at least an entire year.

But for that entire time, it had been a massive, invisible risk - a silent violation of every segmentation principle we were supposed to enforce.

The lesson is that temporary exceptions are rarely ever temporary. In federal environments, they become:

- Permanent attack paths
- Unmonitored bypasses
- Documentation lies
- Audit landmines
- Technical debt with teeth

A temporary exception without tracking is just a permanent vulnerability waiting to be exploited.

## 8\. The Environment Where Everyone Was an Admin

Here's a story about an environment where admin rights were handed out like candy - not because anyone thought it was a good idea, but because everyone was terrified of breaking operations. It's one of those situations every federal NetSecOps operator encounters at least once.

I walked into a new assignment expecting the usual mix of legacy systems and half-finished modernization projects. What I didn't expect was an environment where admin rights were so widespread that least privilege wasn't just ignored - it never even existed to begin with.

It started when I reviewed the local admin group on a random workstation.

There were 14 users in it.

Fourteen.

And that was just the beginning.

**\*\*Least privilege wasn't just ignored - it was never implemented.\*\***

The philosophy seemed to be:

\_"If they need to do their job, give them admin. If they don't need admin, give it to them anyways, you know, just in case."\_

Developers had admin.

Help desk techs had admin.

Contractors had admin.

Even a few interns had admin.

It was like Oprah visited the domain:

"You get admin! You get admin! Everybody gets admin!"

Users were local admins everywhere. Every workstation had a local Administratos group that looked like a small town directory. The justification was always the same:

\_"They need to install their tools."\_

Except "their tools" included:

- Random freeware
- Unsigned utilities
- Personal scripts
- Old versions of Wireshark
- A Minecraft launcher (I wish I were joking)

Local admin wasn't a privilege - it was a lifestyle.

The helpdesk roles were way too powerful. The help desk had:

- Reset password rights for almost everyone
- Unlock account rights for privileged users
- Local admin on all workstations
- Remote access to servers
- Ability to join machines to the domain
- Write access to a GPO used for software deployment

They weren't just your typical help desk - they were junior domain admins with better customer service skills.

Server admins crossed into domain-level access. The server team had a habit of adding themselves to groups "temporarily" to fix issues. But temporary always became permanent.

They had:

- Domain Admin on weekends
- Schema Admin during migrations
- Enterprise Admin "just in case"
- Backup Operators because someone once needed to restore a file
- DNS Admin because they didn't want to open a ticket

Separation of duties wasn't weak - it was nonexistent.

- The database team had access to the servers.
- The server team had access to the domain.
- The network team had access to the firewalls and the servers.
- The app team had access to production.
- The contractors had access to everything the app team had.

It was a perfect circle of "everyone can do everything."

Whenever I asked why someone had admin rights, the answer was always the same:

\_"We can't risk breaking operations."\_

Or

\_"We tried least privilege once and it caused outages."\_

Or my personal favorite out of the mix:

\_"It's faster this way."\_

The fear of downtime outweighed the fear of compromise.

Convenience beat security every time.

The risk created was enormous:

- Any compromised workstation = full domain compromise
- No accountability because everyone had the same level of access
- Impossible incident response because logs showed too many privileged actions
- Privilege creep that had one on for years
- Attack paths everywhere
- No way to enforce FICAM principles which meant we were already out of compliance
- No way to prove compliance

It was a domain built on trust instead of security-enabled controls - and trust is not a security model.

The Lesson

When everyone has admin rights, nobody is secure.

Least privilege isn't a suggestion - it's the only thin standing between a functioning environment and a catastrophic breach.

And the biggest irony?

The fear of "breaking operations" created an environment where a single compromised account could break \_everything.\_

9\. The Tool Found the Issue - But Nobody Wanted to Hear It

Here's a story about one of the most frustrating patterns in federal NetSEcOps: a tool clearly found a real problem - and everyone ignored it because it was too inconvenient. This is another one that still sticks with me because it showed how a perfectly functioning scanner can be defeated by human denial.

We had just rolled out a new AD assessment tool, BloodHound - a tool that is a deep-dive analyzer that maps privileges, ACLs, delegation paths, and attack routes. It was the kind of tool that doesn't care about politics, personalities, semantics, or "how things have always been done." It just reports what it finds, which is the truth.

And the truth it uncovered was real ugly.

The assessment flagged a privilege escalation path that allowed a low-tier help desk user to become a Domain Admin within just three hops.

Three.

The path went like this:

- Help desk user > had write access to a GPO
- That GPO > applied to a server with a misconfigured local admin group
- That server > had a service running as a privileged account
- That service account > was a member of Domain Admins

It was a clean, exploitable chain - the kind attackers dream about.

The tool marked it as Critical.

It was absolutely a real risk. This wasn't noise. This wasn't a false positive. This wasn't theoretical. It was indeed a real, functional attack path that anyone with basic AD knowledge could exploit in minutes.

We validated it manually.

We reproduced it in a lab.

We demonstrated step-by-step.

The risk was real.

And leadership didn't understand it. When we briefed leadership, their reaction was... underwhelming.

They saw the word "help desk" and assumed the tool was exaggerating.

They saw the word "GPO" and assumed it was normal.

They saw the word "service account" and assumed it was harmless.

One manager even went as far as saying \_"If this were really dangerous, we would have heard about it beforehand."\_

That's when I realized the problem wasn't technical - it was cultural.

It was accepted as a "known risk," and instead of fixing it, leadership labeled it  
\*\*\*"Known risk - acceptable for now."\*\*\*

Why?

Because fixing it meant:

- Redesigning GPO structure
- Cleaning up service accounts
- Removing legacy permissions
- Reworking help desk roles
- Updating documentation
- Coordinating across three teams

In other words: \*\*work.\*\*

So they accepted the risk and moved on.

Remediation was delayed... indefinitely

The ticket was marked "deferred."

Then "pending resources."

Then "scheduled for next quarter."

Then "rolled into future modernization."

Then "closed due to inactivity and no movement."

The attack path was left untouched and stayed exactly where it was.

Six months later, during an unrelated incident investigation, we discovered that a contractor workstation had been compromised with malware.

Guess which account the attacker used first?

The help desk account.

Guess which GPO they modified?

The one the tool flagged.

Guess which service account they escalated to?

The privileged one tied to the misconfigured server.

Guess where they ended up?

Domain Admin.

Exactly the path the tool warned about.

The only reason the breach didn't go further was pure luck - the attacker was sloppy and triggered an alert before they could deploy persistence.

But the damage had already been done.

And leadership suddenly cared a lot more about the "known risk."

The Lesson

A tool can identify the problem.

A tool can map the path.

A tool can warn you in bright red letters.

But no tool can force people to act accordingly.

Burying one's head in the sand and ignoring a real finding doesn't make it go away - it just gives an attacker a head start.